

Lecture 8: XV6 Scheduling

Professor G. Sandoval

08



Some slides adapted by G. Sandoval for CS3224, from Tanenbaum & Bo, Modern Operating Systems: 4th ed., (c) 2013 Prentice-Hall, Inc. All rights reserved. Also some Slides by Brendan Dolan-Gavitt and Junfeng Yang @ Columbia.



Today

- Scheduling
- XV6 Scheduling

Scheduling

- To coordinate the running of multiple processes, the OS includes a *scheduling algorithm* that decides what process will be run when, and for how long
- There are tons of different scheduling algorithms with different goals:
 - Maximize CPU utilization
 - Maximize I/O throughput
 - Make the system feel responsive to the user
 - Meet real-time deadlines

When to Schedule

- At timed intervals
 - We can use the hardware *timer interrupt* for this
- When a process exits
 - No process is running now so we must choose one
- When a process blocks on I/O
- When an interrupt happens
 - May signal that I/O is done, and we want to unblock the relevant process

Preemption

- Schedulers can be further classified by whether or not processes can choose when they stop running
- **Non-Preemptive** schedulers let processes run until they block on I/O or yield voluntarily
- **Preemptive** schedulers take advantage of a **timer** interrupt

Class 8: Is the XV6 scheduler preemptive or not?

Preemptive

0%

Non-Preemptive

0%

Hybrid

0%

None of the above

0%

Round Robin Scheduling

- A simple, *preemptive* scheduling algorithm
- Run first process until its *quantum* is used up
- Move that process to the end and run the next process until its quantum is used up
- Simple, fair

Think – Pair – Share

- **Turn to your neighbor:** Why is round robin considered fair?
What's a scenario where it's not ideal?
- 2 mins discussion

Multiplexing

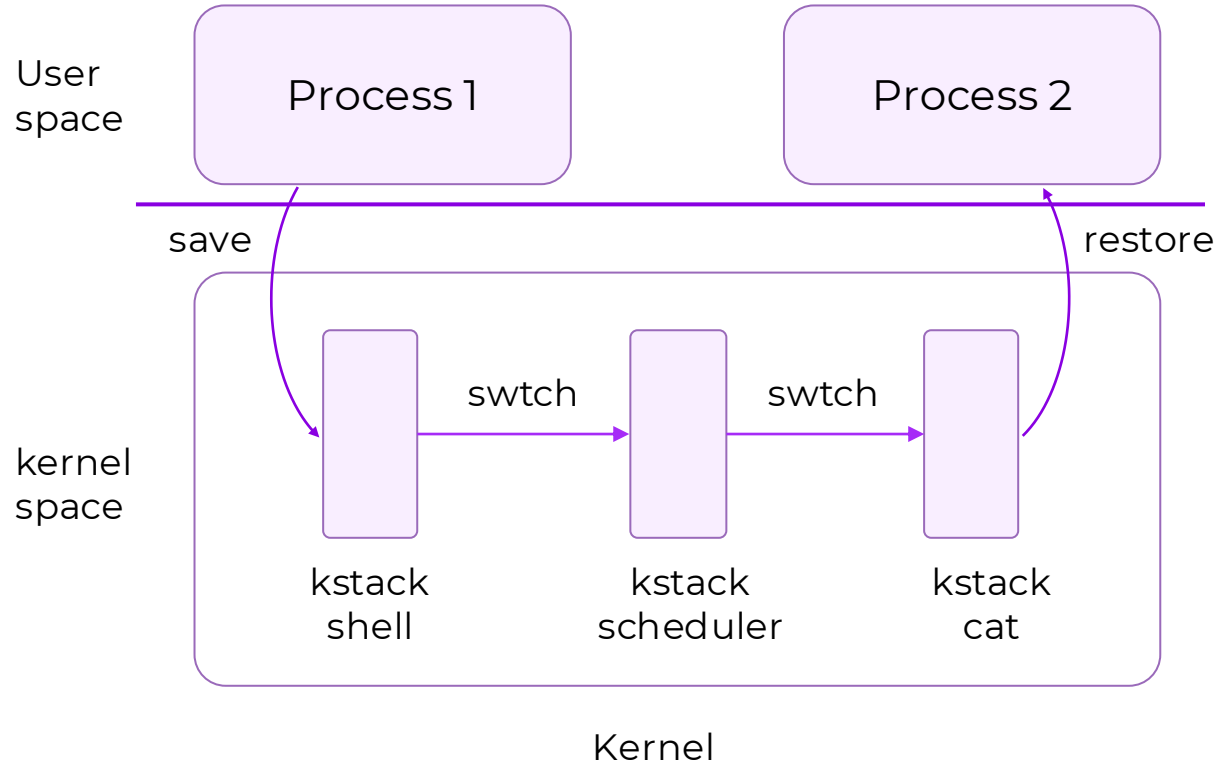
- An operating system is likely to run with more processes than there are processors!
- OS provides the illusion that it has it's own (virtual) processor
- The OS does this by multiplexing **multiple virtual processor** on a **single physical processor**

Multiplexing

- XV6 multiplexes by switching each processor from one process to another in 2 situations:
 - Sleep and wakeup mechanism – wait for device or pipe (I/O)
 - Periodically forces switch when quantum ends

xv6 Scheduler

- xv6 uses *round robin*
- Whenever scheduling happens, *loop over the array* until we find a runnable process and switch to it
- When the timer runs or the process yields, we return to the scheduler loop
- Importantly, we return to the loop at the *exact place* we left off – this is what makes it *fair* and ensures we don't starve processes later in the list



Proc.c

Which line
sets the
process set to
running?

```
1  Void scheduler(void)
2  {
3      struct proc *p;
4      for(;;){
5          // Enable interrupts on this processor.
6          sti();
7          // Loop over process table looking for process to run.
8          acquire(&ptable.lock);
9          for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
10             if(p->state != RUNNABLE)
11                 continue;
12             // Switch to chosen process. It is the process's job
13             // to release ptable.lock and then reacquire it
14             // before jumping back to us.
15             proc = p;
16             switchvm(p);
17             p->state = RUNNING;
18             swtch(&cpu->scheduler, proc->context);
19             switchkvm();
20             // Process is done running for now.
21             // It should have changed its p->state before coming back.
22             proc = 0;
23         }
24         release(&ptable.lock);
25     }
}
```

Proc.c

```
Void scheduler(void)
{
    struct proc *p;
    for(;;) {
        // Enable interrupts on this processor.
        sti();
        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;

            // Switch to chosen process. It is the process's job
            // to release ptable.lock and then reacquire it
            // before jumping back to us.
            proc = p;
            switchvm(p);
            p->state = RUNNING;
            swtch(&cpu->scheduler, proc->context);
            switchkvm();
            // Process is done running for now.
            // It should have changed its p->state before coming back.
            proc = 0;
        }
        release(&ptable.lock);
    }
}
```

[. ..

Switches between kernel &
process

Context Switching

- The actual scheduler context switch (to start executing the process) happens here:
- Let's examine these in more detail

```
// Switch to chosen process.  It is the process's job
// to release ptable.lock and then reacquire it
// before jumping back to us.
proc = p;
switchvm(p);
p->state = RUNNING;
switch(&cpu->scheduler, proc->context);
switchkvm();
```

switchvm

- This function does two things:
 - Switches *the task state segment* to the *user-mode* one
 - Changes the current *virtual address space* to the process's

Task State Segment (TSS)

- The TSS is a special structure defined by x86 intended to help with *context switching*
- *It provides:*
 - Location of **kernel stack** for *this process*
 - Segment descriptor for kernel stack

switchvm

```
// Switch TSS and h/w page table to correspond to process p.
```

```
void
```

```
switchvm(struct proc *p)
```

```
{
```

```
    pushcli();
```

```
    cpu->gdt[SEG_TSS] = SEG16(STS_T32A, &cpu->ts, sizeof(cpu->ts)-1, 0);
```

```
    cpu->gdt[SEG_TSS].s = 0;
```

```
    cpu->ts.ss0 = SEG_KDATA << 3;
```

```
    cpu->ts.esp0 = (uint)proc->kstack + KSTACKSIZE;
```

```
    ltr(SEG_TSS << 3);
```

```
    if(p->pgdir == 0)
```

```
        panic("switchvm: no pgdir");
```

```
    lcr3(v2p(p->pgdir)); // switch to new address space
```

```
    popcli();
```

```
}
```

Set the Stack segment that instructs the hardware to execute system calls and interrupts on the process stack

Crucial instruction for setting the VM Context. Sets the CR3 register to the physical of the page dir of process p.

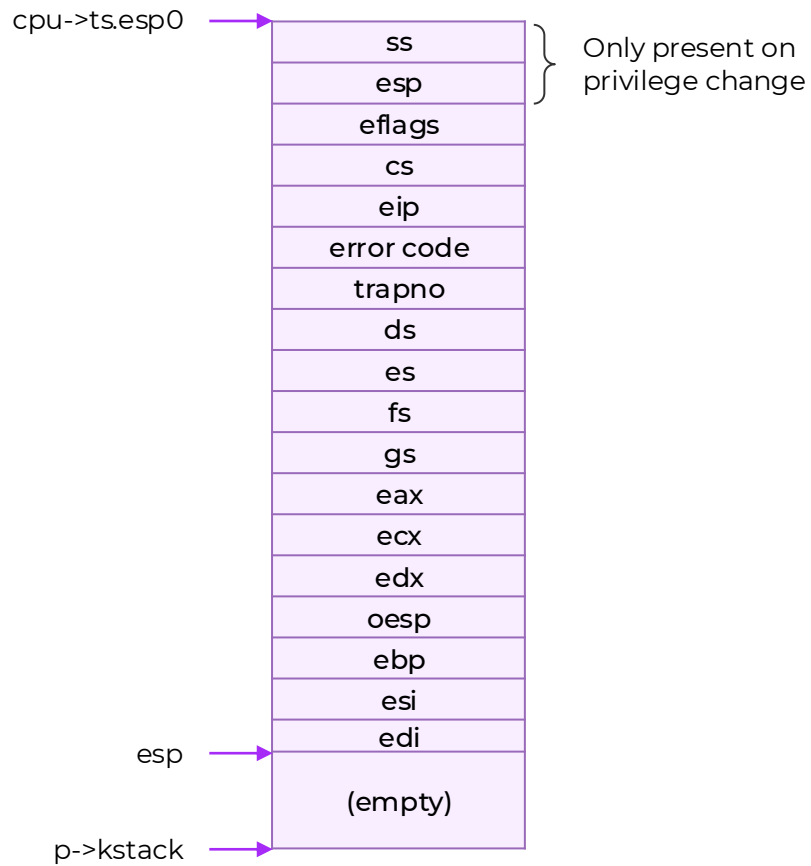
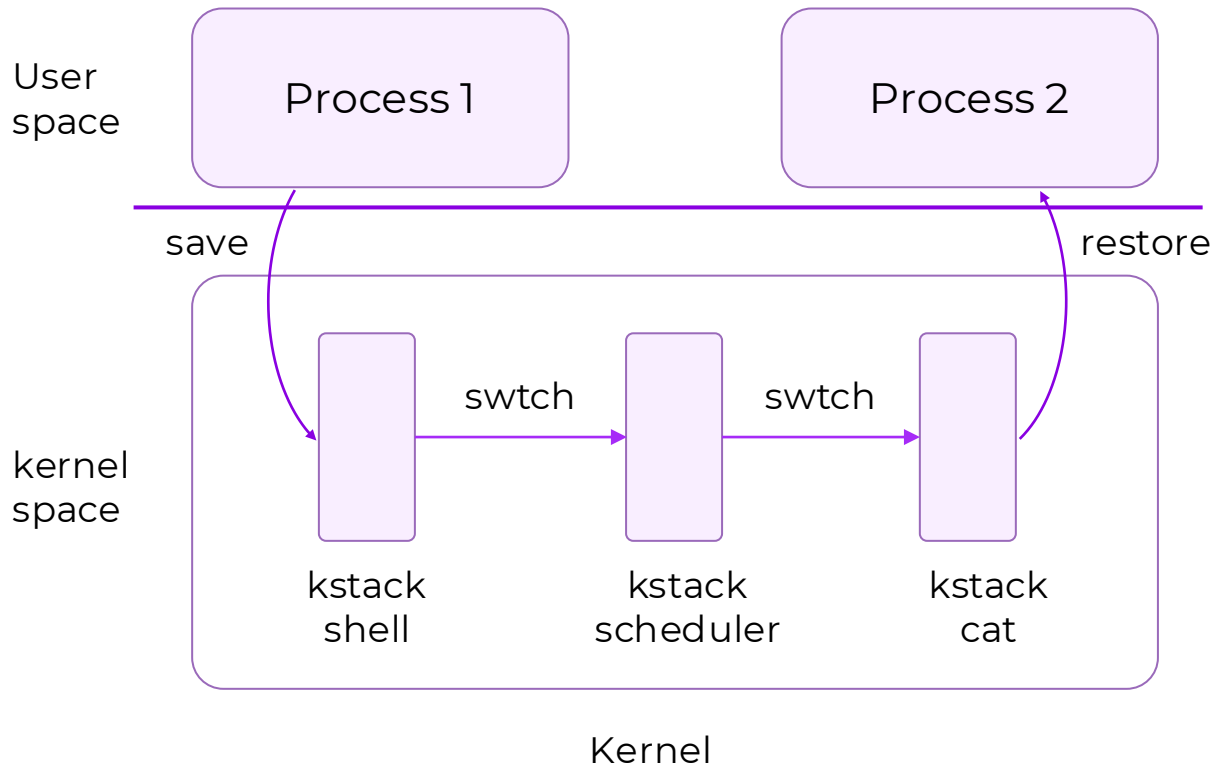


Figure 3-2. The trapframe on the kernel stack

switch



```

void
scheduler(void)
{
    struct proc *p;
    for(;;) {
        // Enable interrupts on this processor.
        sti();
        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
            if(p->state != RUNNABLE)
                continue;
            // Switch to chosen process. It is the process's job
            // to release ptable.lock and then reacquire it
            // before jumping back to us.
            proc = p;
            switchvm(p);
            p->state = RUNNING;
            swtch(&cpu->scheduler, proc->context);
            switchkvm();
            // Process is done running for now.
            // It should have changed its p->state before coming back.
            proc = 0;
        }
        release(&ptable.lock);
    }
}

```

switch

- Does the actual work of switching between the **kernel scheduler context** and the **kernel process context**.
- Only needs to save a few registers :

```
struct context {  
    uint edi;  
    uint esi;  
    uint ebx;  
    uint ebp;  
    uint eip;  
};
```
- What about the others? What about eax?

What About the Others?

- Don't need to save segment registers because those are the same for all kernel contexts
- Don't need to save `eax`, `ecx`, `edx` – in the gcc calling convention, these are not assumed to persist across function calls
- So any code that calls `switch()` will automatically preserve them

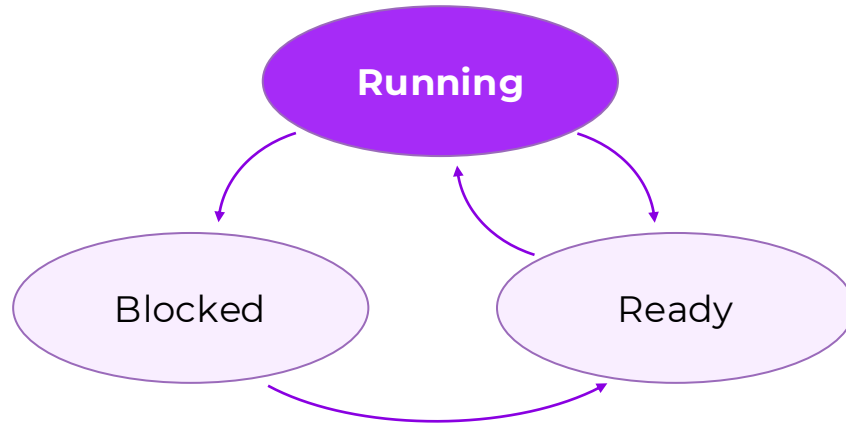
```
# Context switch
#
# void swtch(struct context **old, struct context *new);
#
# Save current register context in old
# and then load register context from new.
```

```
.globl swtch
swtch:
    movl 4(%esp), %eax
    movl 8(%esp), %edx
    # Save old callee-save registers
    pushl %ebp
    pushl %ebx
    pushl %esi
    pushl %edi
    # Switch stacks
    movl %esp, (%eax)
    movl %edx, %esp
    # Load new callee-save registers
    popl %edi
    popl %esi
    popl %ebx
    popl %ebp
    ret
```

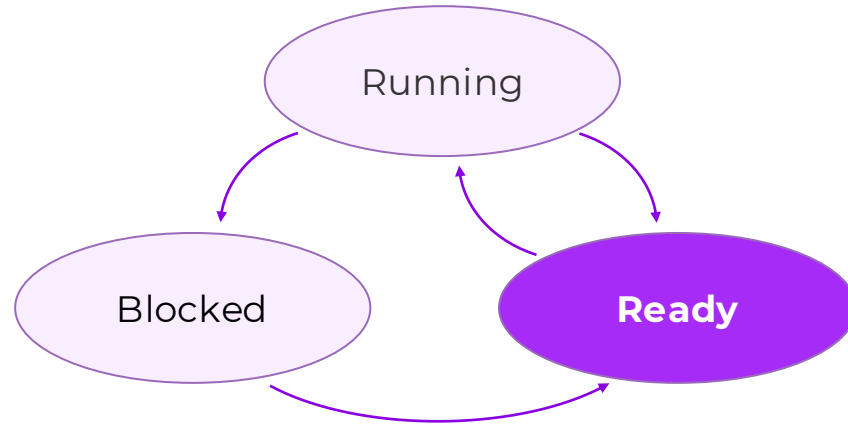
Note: EIP is implicitly saved by executing the call
– call pushes the EIP of the next instruction onto the stack

```
struct context {
    uint edi;
    uint esi;
    uint ebx;
    uint ebp;
    uint eip;
};
```

Now process is running



Switching Back (Finished Quantum)



switch-ing Back

- When a process "gives up" the CPU, yield() is called
- yield() makes the process runnable and then calls sched()
 - Note: *not* the same as scheduler()

```
// Give up the CPU for one scheduling round.  
void  
yield(void)  
{  
    acquire(&ptable.lock); //DOC: yieldlock  
    proc->state = RUNNABLE;  
    sched();  
    release(&ptable.lock);  
}
```

sched()

```
// Enter scheduler. Must hold only ptable.lock
// and have changed proc->state.
void
sched(void)
{
    int intena;
    if(!holding(&ptable.lock))
        panic("sched ptable.lock");
    if(cpu->ncli != 1)
        panic("sched locks");
    if(proc->state == RUNNING)
        panic("sched running");
    if(readeflags() & FL_IF)
        panic("sched interruptible");
    intena = cpu->intena;
    swtch(&proc->context, cpu->scheduler);
    cpu->intena = intena;
}
```

Where does this go?

Here

```
void
scheduler(void)
{
    struct proc *p;
    for(;;) {
        // Enable interrupts on this processor.
        sti();
        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++) {
            if(p->state != RUNNABLE)
                continue;
            // Switch to chosen process. It is the process's job
            // to release ptable.lock and then reacquire it
            // before jumping back to us.
            proc = p;
            switchvm(p);
            p->state = RUNNING;
            swtch(&cpu->scheduler, proc->context);
            switchkvm();
            // Process is done running for now.
            // It should have changed its p->state before coming back.
            proc = 0;
        }
        release(&ptable.lock);
    }
}
```

Class 8: My understanding of the scheduler is reflected on this face. (26)

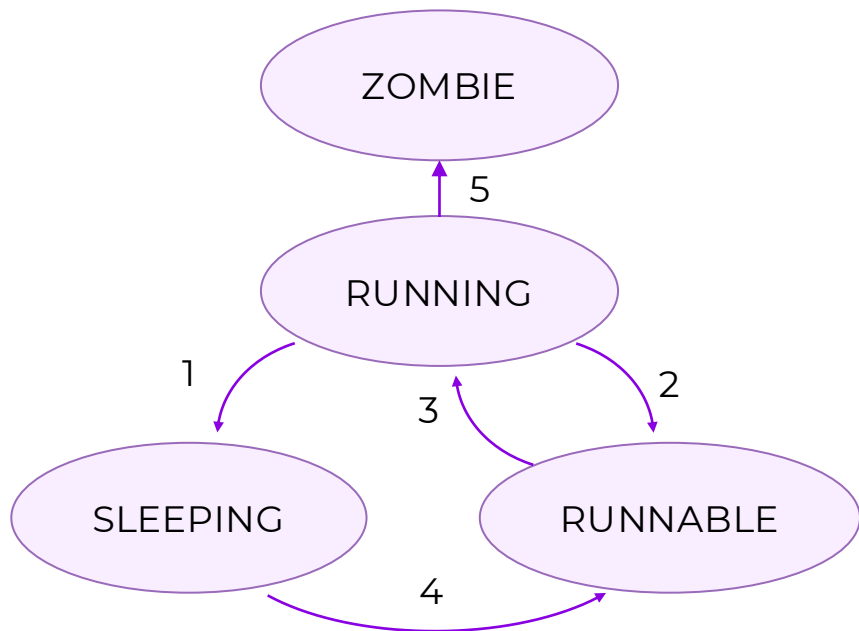




Today

- Scheduling
- **More XV6 Scheduling**

xv6: State Transitions



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available
5. Process Exits

1. Process Blocks for Input

RUNNING → SLEEPING

- Example: waiting for the user to type something
- In `console.c`, `consoleread` calls `sleep()` to put any process waiting for keyboard input to sleep until a key is pressed

```
int
consoleread(struct inode *ip, char *dst, int n)
{
    uint target;
    int c;
    iunlock(ip);
    target = n;
    acquire(&input.lock);
    while(n > 0){
        while(input.r == input.w){
            if(proc->killed){
                release(&input.lock);
                ilock(ip);
                return -1;
            }
            sleep(&input.r, &input.lock);
        }
    }
}
```

```
// Atomically release lock and sleep on chan.
// Reacquires lock when awakened.
void
sleep(void *chan, struct spinlock *lk)
{
    [... locking code omitted ...]
    // Go to sleep.
    proc->chan = chan;
    proc->state = SLEEPING;
    sched(); ←
    // Tidy up.
    proc->chan = 0;
    [... code omitted ...]
}
```

Remember that sched()
switches from process back to
the kernel

2. Scheduler Picks Another Process

RUNNING → RUNNABLE

- Example: time slice is up (timer interrupt fires)
- In trap.c, the trap function will forcibly give up the CPU on a timer interrupt

Trap

```
void
trap(struct trapframe *tf)
{
    [... other trap handling omitted ...]
    // Force process to give up CPU on clock tick.
    // If interrupts were on while locks held, would need to check nlock.
    if(proc && proc->state == RUNNING && tf->trapno == T_IRQ0+IRQ_TIMER)
        yield();
    // Check if the process has been killed since we yielded
    if(proc && proc->killed && (tf->cs&3) == DPL_USER)
        exit();
}
```

Gives up the CPU

Yielding

```
// Give up the CPU for one
// scheduling round.
void
yield(void)
{
    acquire(&ptable.lock); //DOC: yieldlock
    proc->state = RUNNABLE;
    sched();
    release(&ptable.lock);
}
```

3. Scheduler Picks This Process

RUNNABLE → RUNNING

- Example: the main scheduler() function in proc.c

```

Void scheduler(void)
{
    [ . . . ]
    for(;;){
        // Enable interrupts on this processor.
        sti();
        // Loop over process table looking for process to run.
        acquire(&ptable.lock);
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->state != RUNNABLE)
                continue;
            // Switch to chosen process. It is the process's job
            // to release ptable.lock and then reacquire it
            // before jumping back to us.
            proc = p;
            switchvm(p);
            p->state = RUNNING;
            swtch(&cpu->scheduler, proc->context);
            switchkvm();
            // Process is done running for now.
            // It should have changed its p->state before coming back.
            proc = 0;
        }
        release(&ptable.lock);
    }
}

```

4. Input Becomes Available

SLEEPING → RUNNABLE

- Example: user presses a key
- In trap.c, the interrupt handler recognizes that a keyboard interrupt has occurred and calls the keyboard handler, which calls the console handler
- Console handler finally calls wakeup() to notify any processes waiting for keyboard input

trap.c

```
void
trap(struct trapframe *tf)
{
    [...]
    switch(tf->trapno) {
    [...]
    case T_IRQ0 + IRQ_KBD:
        kbdintr();
    [...]
    }
}
```

kbd.c

```
void
kbdintr(void)
{
    consoleintr(kbdgetc);
}
```

console.c

```
void
consoleintr(int (*getc)(void))
{
    [...]
    if(c == '\n' || c == C('D') || input.e == input.r+INPUT_BUF){
        input.w = input.e;
        wakeup(&input.r);
    }
    [...]
}
```

proc.c

```
// Wake up all processes sleeping on chan.
void
wakeup(void *chan)
{
    acquire(&ptable.lock);
    wakeup1(chan);
    release(&ptable.lock);
}
// Wake up all processes sleeping on chan.
// The ptable lock must be held.
static void
wakeup1(void *chan)
{
    struct proc *p;

    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++)
        if(p->state == SLEEPING && p->chan == chan)
            p->state = RUNNABLE;
}
```

5. Process Exits

RUNNING → ZOMBIE

- Happens at process exit (i.e. the `exit()` system call)
- Process gets marked as a **zombie**, and anyone that might be waiting for its exit status gets woken up
- Process doesn't actually get destroyed until someone calls `wait()`
- If the parent isn't around to call `wait()`, the zombie gets reparented (assigned to `init`)

proc.c

```
// Exit the current process. Does not return.
// An exited process remains in the zombie state
// until its parent calls wait() to find out it exited.
void
exit(void)
{
    struct proc *p;
    [...]
    // Parent might be sleeping in wait().
    wakeup1(proc->parent);

    // Pass abandoned children to init.
    for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
        if(p->parent == proc){
            p->parent = initproc;
            if(p->state == ZOMBIE)
                wakeup1(initproc);
        }
    }

    // Jump into the scheduler, never to return.
    proc->state = ZOMBIE;
    sched();
    panic("zombie exit");
}
```

Who Cleans Up?

```
// Wait for a child process to exit and return its pid.
// Return -1 if this process has no children.
int
wait(void)
{
    struct proc *p;
    int havekids, pid;
    acquire(&ptable.lock);
    for(;;){
        // Scan through table looking for zombie children.
        havekids = 0;
        for(p = ptable.proc; p < &ptable.proc[NPROC]; p++){
            if(p->parent != proc)
                continue;
            havekids = 1;
            if(p->state == ZOMBIE){
                // Found one.
                pid = p->pid;
                kfree(p->kstack);
                p->kstack = 0;
                freevm(p->pgdir);
                p->state = UNUSED;
                p->pid = 0;
                p->parent = 0;
                p->name[0] = 0;
                p->killed = 0;
                release(&ptable.lock);
                return pid;
            }
        }
    }
    [...]
    // Wait for children to exit. (See wakeup1 call in proc_exit.)
    sleep(proc, &ptable.lock); //DOC: wait-sleep
}
}
```

The Role of Init

- Init is the first process created
- It spawns the system shell (sh)
- After starting the shell, sits in a loop calling `wait()` in case any zombies get assigned to it

init.c

```
int
main(void)
{
    int pid, wpid;
    [...]
    for(;;){
    [...]
        while((wpid=wait()) >= 0 && wpid != pid)
            printf(1, "zombie!\n");
    }
}
```

Question

- How can we create an orphan zombie that must be adopted by init?
- Or, restated – how do we make init.c reach the printf that prints "zombie!" ?

zombie.c

```
// Create a zombie process that
// must be reparented at exit.
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(void)
{
    if(fork() > 0)
        sleep(5); // Let child exit before parent.
    exit();
}
```



```
QEMU
SeaBIOS (version rel-1.7.5-0-ge51488c-20140602_164612-nilsson.home.kraxel.org)

iPXE (http://ipxe.org) 00:03.0 C980 PCI2.10 PnP PMM+1FF93BB0+1FEF3BB0 C980

Booting from Hard Disk...

cpu0: starting xv6

cpu1: starting
cpu0: starting
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star
t 58
init: starting sh
$ zombie
zombie!
$ _
```